

前言

该文章是针对于图像生成任务提出的，但是其思想具有启发性。

1. 虽然transformer的注意力机制非常有效，但是随着规模的扩大，计算量增长可怕，计算复杂度是 $O(N \times N)$
 2. 注意力具有全局性，这是复杂度很高的原因，但是在生成图像的任务之中，并不是所有的地方都需要观看到全局的信息来补全图像，相反，只有少数，需要，其余多数是纹理，质地等信息，只需要观看局部信息就能够完成
- 所以，全局的信息提取（特别是多头注意力的全局性）具有冗余性，需要引入稀疏性算法。

稀疏注意力

下面给出文章中提出的稀疏注意力

Strided Sparse Transformer 和 Fixed Sparse Transformer 示意图

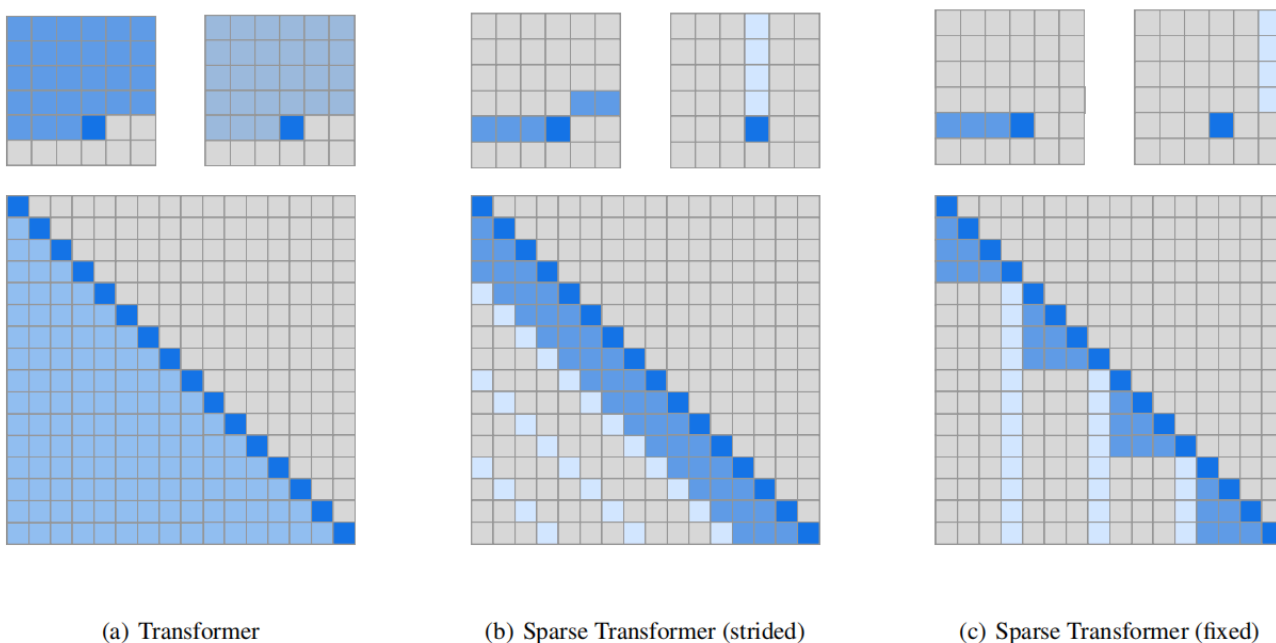


Figure 3. Two 2d factorized attention schemes we evaluated in comparison to the full attention of a standard Transformer (a). The top row indicates, for an example 6x6 image, which positions two attention heads receive as input when computing a given output. The bottom row shows the connectivity matrix (not to scale) between all such outputs (rows) and inputs (columns). Sparsity in the connectivity matrix can lead to significantly faster computation. In (b) and (c), full connectivity between elements is preserved when the two heads are computed sequentially. We tested whether such factorizations could match in performance the rich connectivity patterns of Figure 2.

1. Strided固定了步幅，每一个token都会固定关注到固定长度s的上文，同时，他会选取每一个间隔s的token也计算（直观上是每一列的token），通常情况下，s取值 $\sqrt{n}$ ，n是图片的长宽。
2. 固定了一个块的长度，每一个token只会关注到块内的内容，块的长度通常和固定长度一致。还每一个块的第一个token会关注到右边全局，使得第一个块能够收集到全局的信息，不会引导后面的输出走向畸形。

Strided ST

这是研究的重点，在稀疏性的同时能够看到全局的特征

第 1 个注意力头（局部头）：

关注当前位置附近的局部窗口，即

$$A_i^{(1)} = \{t, t+1, \dots, i\} \quad t = \max(0, i-l)$$

l 为步幅，通常取 $\sqrt{n}$ ，捕捉局部依赖；

第 2 个注意力头（步幅头）：

关注按步幅间隔的位置，即

$$A_i^{(2)} = \{j \mid (i-j) \bmod l = 0\}$$

跨越局部窗口覆盖远距离位置；

Fixed ST

第 1 个注意力头（块内头）：

关注当前位置所在的“块”内所有位置，即

$$A_j^{(1)} = \{\min(i \mid \lfloor j/l \rfloor = \lfloor i/l \rfloor) : j\}$$

(l 为块大小)，聚焦块内局部依赖；

第 2 个注意力头（块间头）：

关注前一个块的固定子区域，即

$$A_j^{(2)} = \{j \mid j \bmod l \in \{t, \dots, l\}\} \quad (t = l - c)$$

c 为超参数，通常取 8/16/32)，确保块间信息传递；

注意力结构

自注意力

自注意力：

$$ATTN(X, S) = (a(x_i, s_i)) \quad \{i \in n\}$$

$$a(x_i, s_i) = \text{Softmax}\left(\frac{W_q x_i (W_k x_j)^T}{\sqrt{d}}\right) W_v x_j \quad \{j \in S_i\}$$

s_i 是 x_i 对应的需要注意力的 X 的子集

- 所有单词都要生成 Q ，每一个 q 都要对应他的注意力集合 s 来计算注意力。

多头注意力

与传统的多头注意力计算所有token注意力相比，稀疏注意力赋予了多头注意力的多功能性，也就是不同的头使用不同的稀疏模式。

并且有

$$\|S_i\| \text{与} \sqrt[p]{n} \text{成正比}$$

p 是多头注意力的头数，这也符合我们提出的两种注意力模式。

所有的头的 S_i U起来，最佳的情况是覆盖了所有的token，也就是在稀疏的同时保存全局性

传统的多头会被认为处理不同的问题，和CNN的通道的思想一致

这里的多头注意力针对于图像生成，不同的头查看了图像不同的地方，也有不同的头处理不同问题的思想存在

这里的稀疏注意力舍弃了注意力的个体全局性，但是通过多头保留了总体的全局性（全局性的程度取决于你的设计），很有创新性

文章中给出了三种多头注意力的堆放模式，也就是文章对于**多头**有不同的理解

方法一

在一个decoder之中，FFN之前的注意力变成多个残差块，每一个残差块对应一种类别的稀疏注意力，这个时候 W_q, W_k, W_v 在每一个残差块之中都不一样

每一个头是按顺序堆叠的，每一个残差块的公式：

$$Attention(X, S)^{(r)} = W_p \cdot ATTN(X, S^{(r \bmod p)})$$

- r 是残差块的序号， p 是多头注意力的头数，这标志了第 r 个残差块使用的注意力类型指引

方法二

这个方法比较密集，他关注的子集是多个头直接U起来的，虽然是稀疏，但是由于我们靠近全局性的设计，他计算量偏大，稀疏性表现弱。

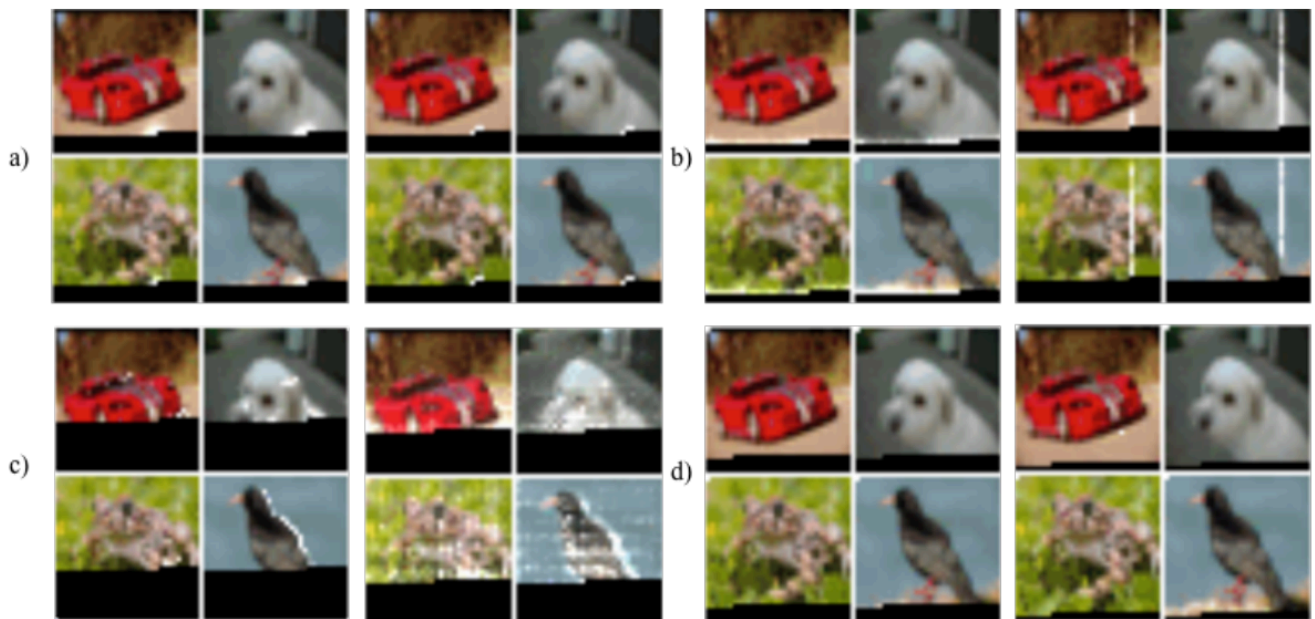
$$Attention(X, S) = W_p \cdot ATTN(X, \cup_{m=1}^p S^{(m)})$$

方法三

也就是我们想象的，传统的多头注意力的改观。

$$Attention(X, S) = W_p \cdot \left(ATTN(X, S^{(m)}) \right) \quad \{m \in p\}$$

可视化



上面的图片可视化了稀疏注意力

(白色表示被当前生成像素关注的地方，黑色表示被掩码，未生成的地方)

- a) 关注局部区域，仅仅需要纹理等信息就能够生成的像素
- b) 关注行和列信息，这是稀疏注意力的表现
- c) 关注到全局的信息，在生成关键区域的时候，稀疏注意力也不会丧失关注全局的能力
- d) 注意力具有高稀疏性，说明需要关注的信息（全局信息非常少）。

通过上述例子，我们直观看到

- 1. 并不是所有场景都需要全局的注意力
- 2. 稀疏注意力不会丧失关注全局的能力

总结

文章创造性提出了两种稀疏性注意力的概念，并且通过实验，提出了三种适用于改造 transformer 的堆叠办法。

减少了注意力的运算量，将复杂度从 $O(n^2)$ 降低到 $O(n \cdot p \sqrt{N})$ 。

虽然我不太认为它能够直接嫁接到处理文本的大模型之中，但是稀疏的思想能够启发我们在注意力之中设置一个上下文窗口，减少全局性的无用注意。